



ECE 401/501c Final Project:

Vinyl Record Library + Online Store

Travis Potter, Andrew Kostick, Ricardo Ramirez



Vinyl Record Library + Online Store

Problem We are Trying to Solve:

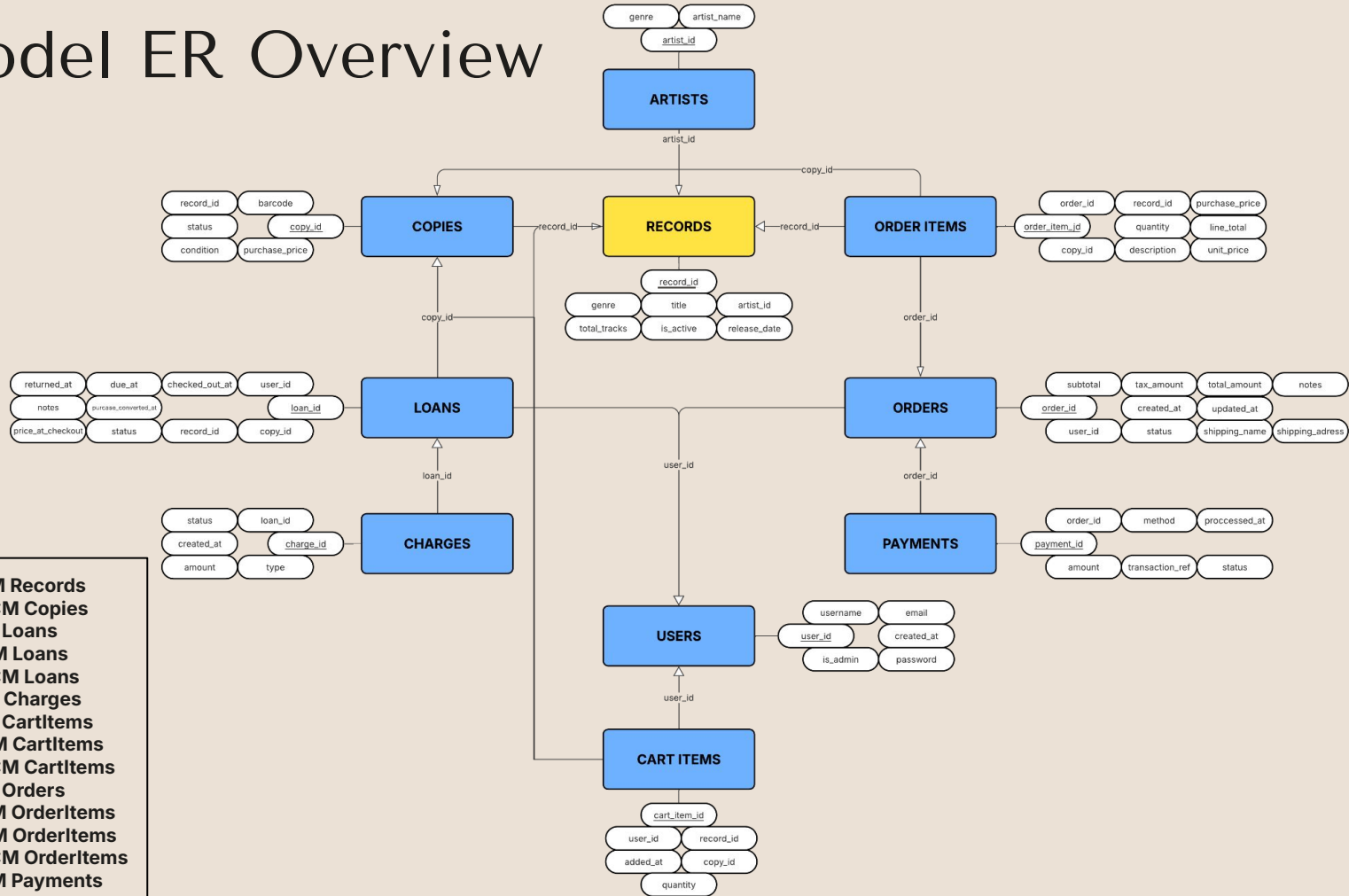
Enable both library-style borrowing *and* online purchasing

Our Goal:

Build a unified system that supports:

- Library workflows: checkout, return, availability tracking
- Store workflows: pricing, shopping cart, purchase checkout
- Search-based navigation across Artists/Records
- Accurate tracking of physical copies
- Transactional correctness for both loans and purchases

Model ER Overview



Artists 1:M Records
 Records 1:M Copies
 Users 1:M Loans
 Copies 1:M Loans
 Records 1:M Loans
 Loans 1:M Charges
 Users 1:M CartItems
 Copies 1:M CartItems
 Records 1:M CartItems
 Users 1:M Orders
 Orders 1:M OrderItems
 Copies 1:M OrderItems
 Records 1:M OrderItems
 Orders 1:M Payments

```

-- =====
-- User management
-- =====
CREATE TABLE IF NOT EXISTS Users (
  user_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  username   TEXT NOT NULL UNIQUE,
  email      TEXT NOT NULL UNIQUE,
  is_admin   INTEGER NOT NULL DEFAULT 0 CHECK (is_admin IN (0,1)),
  password   TEXT NOT NULL,
  created_at TEXT DEFAULT (datetime('now'))
);

-- =====
-- Physical copies
-- =====
CREATE TABLE IF NOT EXISTS Copies (
  copy_id    INTEGER PRIMARY KEY AUTOINCREMENT,
  record_id  INTEGER NOT NULL,
  barcode    TEXT NOT NULL UNIQUE,
  purchase_price NUMERIC(10,2) DEFAULT 0,
  condition  TEXT DEFAULT 'GOOD' CHECK (condition IN ('NEW','GOOD','FAIR','POOR','DAMAGED')),
  status     TEXT NOT NULL DEFAULT 'AVAILABLE' CHECK (status IN ('AVAILABLE','CHECKED_OUT','SOLD')),
  FOREIGN KEY (record_id) REFERENCES Records(record_id) ON DELETE CASCADE
);

```

```

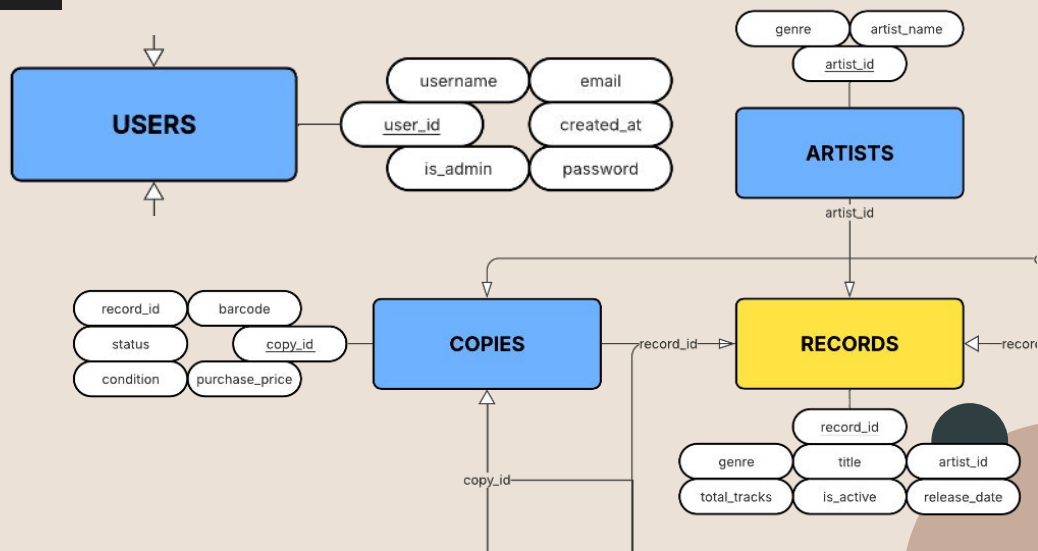
CREATE TABLE IF NOT EXISTS Artists (
  artist_id INTEGER PRIMARY KEY AUTOINCREMENT,
  artist_name TEXT NOT NULL UNIQUE,
  genre TEXT
);

CREATE TABLE IF NOT EXISTS Records (
  record_id INTEGER PRIMARY KEY AUTOINCREMENT,
  artist_id INTEGER,
  title TEXT NOT NULL,
  genre TEXT,
  is_active INTEGER DEFAULT 1,
  release_date TEXT,
  total_tracks INTEGER,
  FOREIGN KEY (artist_id) REFERENCES Artists(artist_id)
);

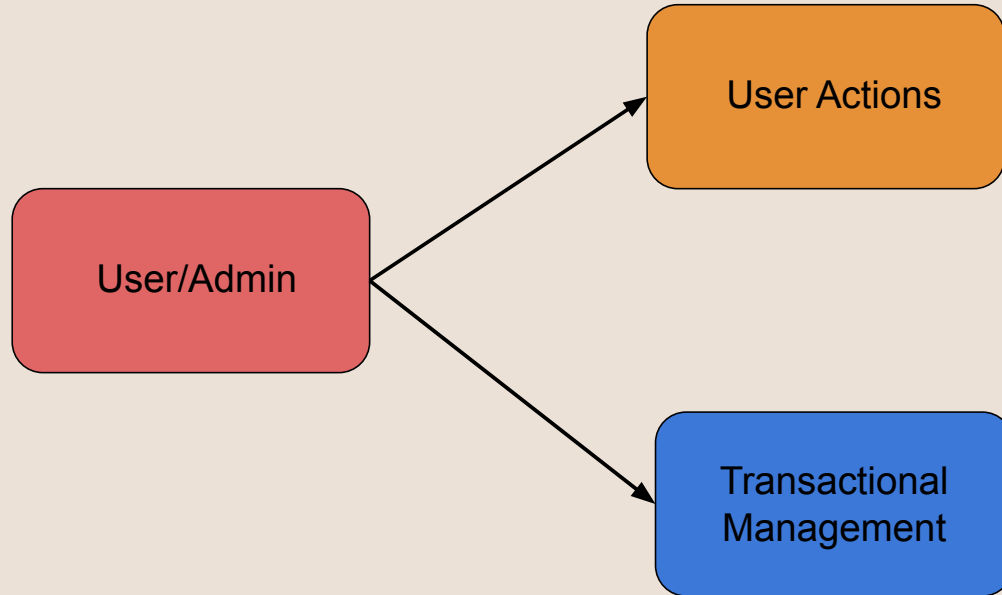
```

KEY RELATIONS

- ★ Users 1:M Loans
- ★ Users 1:M Orders
- ★ Records 1:M Copies
- ★ Copies M:1 Loans



Workflow



USER ACTIONS

- Login User/Admin
- Register User
- Search Collection/Catalog
- Checkout/Return Records
- Add items to Cart
- Purchase Records
- View Loan & Purchase History

TRANSACTIONAL MANAGEMENT

- Record Copy Status
 - ◆ Available
 - ◆ Checked Out
 - ◆ Returned
 - ◆ Purchased
- View Cart Items
- View Purchase History

QUICK DEMO



Results:

- Library workflows: checkout, return, availability tracking
- Search-based navigation across Artists/Records
- Store workflows: pricing, shopping cart, purchase checkout
- Accurate tracking of physical copies
- Transactional correctness for both loans and purchases



Purchasing Algorithm

Steps	Action Detail	SQL PSEUDO Operations
1. Data Retrieval	User_id used to identify items in cart.	SELECT user_id, username, email FROM Users WHERE user_id = ? AS <i>user info</i> SELECT <i>copy attributes</i> FROM CartItems JOIN <i>user info</i>
2. Inventory Check	Validates that each copy is still AVAILABLE and has a price.	SELECT <i>copy attributes</i> FROM CartItems WHERE <i>copy.status</i> = "AVAILABLE"
3. Order Creation	1. Creates one row in Orders. 2. Creates one row per item in OrderItems. 3. Creates one simulated Payment.	INSERT INTO Orders (...); INSERT INTO OrderItems (...); INSERT INTO Payments (...);
4. Inventory Update	Marks the purchased Copies as SOLD.	UPDATE <i>copy</i> SET status = 'SOLD' WHERE <i>copy_id</i> = ? IN CartItems
5. Cleanup	Clears the cart.	DELETE * FROM CartItems WHERE user_id = ?;
6. Output	Prints a clear summary.	SELECT * FROM Orders WHERE user_id = ?;

Query Optimizations

1. Semi - 3NF

- a. Reduction of duplicates in data → smaller, well-structured tables
- b. Relevant Queries Improved: Joins & Updates
- c. Examples: Separation of tables *Orders* & *OrderItems*

2. Indexing

- a. Creates system of rapid lookups for high-use queries
- b. Relevant Queries Improved: Select, Join, & Where
- c. Example: Index on *Copies(record_id)* reduces fetch time for all copies of a record

OVERALL IMPROVEMENT: faster queries, efficient joins, and scalable performance of the system

```

-- =====
-- E-commerce indexes
-- =====
CREATE INDEX IF NOT EXISTS idx_cartitems_user ON CartItems(user_id, added_at);
CREATE INDEX IF NOT EXISTS idx_orders_user_created ON Orders(user_id, created_at);
CREATE INDEX IF NOT EXISTS idx_orderitems_order ON OrderItems(order_id);
CREATE INDEX IF NOT EXISTS idx_payments_order ON Payments(order_id, processed_at);

-- =====
-- Indexes
-- =====
→ CREATE INDEX IF NOT EXISTS idx_copies_record ON Copies(record_id);
CREATE INDEX IF NOT EXISTS idx_copies_status ON Copies(status);
CREATE INDEX IF NOT EXISTS idx_loans_user ON Loans(user_id, returned_at);
CREATE INDEX IF NOT EXISTS idx_loans_copy ON Loans(copy_id, returned_at);
CREATE INDEX IF NOT EXISTS idx_loans_due ON Loans(due_at, returned_at);
CREATE INDEX IF NOT EXISTS idx_charges_loan ON Charges(loan_id);
CREATE INDEX IF NOT EXISTS idx_charges_status ON Charges(status, created_at);

```

Example: "Show me all records whose artist has 'Miles' in their name, sorted nicely."

Without Indexing

- `WHERE a.artist_name LIKE '%Miles%'`
- `JOIN Artists a ON r.artist_id = a.artist_id`
- `ORDER BY artist_name, r.title`

Every row in *Artists* is scanned.

Run time = $O(N)$

With Indexing

- `WHERE a.artist_name LIKE '%Miles%'`
- `JOIN Artists a ON r.artist_id = a.artist_id`
- `ORDER BY artist_name, r.title`

Database uses **artist_name** index, moving to first artist like "Miles". Only rows of the subset identified by the index are scanned in *Artist*.

Run Time = $O(\log N)$

```
PS C:\Users\Ricardo\Desktop\ECE501\HybridLibraryRecordStore> python src/benchmark_queries.py
=== No indexes (data/main_no_idx.db) ===
Artist search: 22.38 ms for 200 runs
Availability: 13.74 ms for 200 runs

=== With indexes (data/main_v2.db) ===
Artist search: 13.50 ms for 200 runs
Availability: 7.45 ms for 200 runs
```

Existing System Comparisons

- Hierarchical Ordering
 - Hierarchical Ordering is organizing data with parent-child relationships
 - Could be implemented to improve search and filtering performance
- Decentralized Database
 - Decentralized is a system distributed among many locations
 - Would be hard to implement because it requires lots of hardware and computing
 - Would allow for simultaneous access to interface from multiple users
- Machine Learning
 - Could be implemented with recommending records to users
- "The design and implementation of a digital music library: Framework, data model, query language, and indices of HIT-DML" - Wang, C., Li, J., & Shi, S. (2006)
- "Towards an Open and Scalable Music Metadata Layer" - Hardjono, T., Howard, G., Scace, E., Chowdhury, M., Novak, L., Gaudet, M., Anderson, J., d'Avis, N., Kulis, C., Sweeney, E., & Vaughan, C. (2019).
- "The Music Streaming Sessions Dataset" - Brost, B., Mehrotra, R., & Jehan, T. (2019).

The End

Thank you for listening

Questions?

